DOCKER

A Visual Guide for Absolute Beginners

"It works on my machine." - Docker fixes that.

01 The Problem

02 What is Docker?

03 Images vs Containers

04 How It Actually Works

05 The Docker Lifecycle

06 Commands Cheat Sheet

07 Dockerfile Explained

08 Volumes & Networks

09 Docker Compose

10 Best Practices

SECTION 01

The Problem Docker Solves

You write code on your laptop. It works perfectly. You send it to a friend. It crashes. They send it to the server. It crashes differently. Sound familiar?

Environment	What happens
Your Laptop	Python 3.11, macOS, lib v2.4 -> **Works!**
Friend's Laptop	Python 3.9, Windows, lib v1.8 -> **Crashes**
Server	Python 3.12, Ubuntu, lib v3.0 -> **Crashes**

What actually goes wrong

Different OS

macOS, Windows, and Linux handle files, paths, and permissions differently.

Different versions

Python 3.9 vs 3.12 have real breaking changes. Your friend's machine is not yours.

Missing libraries

A dependency installed on your laptop may be absent or a different version elsewhere.

The fix. Ship the app *together* with its OS, runtime, libraries, and config as one sealed bundle. That bundle is a **container**. Docker is the tool that builds, ships, and runs containers.

SECTION 02

What is Docker?

Docker packages your app - with everything it needs to run - into a **container**. That container runs identically on any machine that has Docker.

THE BLUEPRINT

Image

A read-only template. Contains your app + everything it needs. Built once, never changes.

Think: a recipe card.

THE RUNNING THING

Container

A running instance of an image. It's alive, has state, and you can start, stop, or delete it.

Think: a cooked meal from that recipe.

Simple analogy. An **image** is a class in object-oriented programming. A **container** is an instance of that class. One image can spawn many containers.

Without Docker vs With Docker

Setup	What happens
Without	App A needs lib v2.4. App B needs lib v1.8. They conflict on the same machine. Upgrading one breaks the other.
With	Each app lives in its own container, carrying its own libraries. No conflict. No dependency hell.

SECTION 03

How Docker Actually Works

Docker doesn't simulate a whole computer (like a virtual machine does). It *shares the host's kernel* and isolates each container with Linux namespaces and cgroups. That's why containers start in milliseconds, not minutes.

Approach	Cost and speed
Virtual Machine	Full guest OS copy per app. Gigabytes. Boots in minutes. Heavy.
Docker Container	Shares host kernel. Megabytes. Starts in milliseconds. Light.

Why containers are so fast

- **No full OS copy.** Containers share the host kernel. VMs each carry a whole operating system - that's gigabytes of overhead per VM.
- **Namespaces.** Linux isolates each container's view of the filesystem, network, and processes. Each container thinks it's alone on the machine.
- **cgroups.** Linux limits how much CPU and memory each container can use. One runaway container can't starve the others.
- **Layer caching.** Images are built in layers. Change one line of your app, and only the last layer is rebuilt - everything below is reused instantly.

Important consequence. Containers are *ephemeral* - if you delete one, anything written inside it is gone. That's why we use **volumes** (page 8) for anything we want to keep.

SECTION 04

The Docker Lifecycle

Every Docker project follows the same lifecycle. Understand this once, and you understand Docker.

#	Step	What it means
1	Dockerfile	You write instructions (a recipe).
2	Image	Build produces a sealed read-only blueprint.
3	Registry	Push the image to Docker Hub or a private registry.
4	Container	Pull it anywhere and run it - it starts identically on any host.

The four commands that matter most

```
docker build -t myapp:1.0 .    # 1 -> 2:  Dockerfile -> Image
docker push  myapp:1.0         # 2 -> 3:  Image -> Registry
docker pull  myapp:1.0         # 3 -> 4:  Registry -> your machine
docker run   myapp:1.0         # 4:      Image -> running Container
```

Mental model. Docker is a *shipping pipeline* for your code. Write a Dockerfile (the recipe), build it into an image (the frozen meal), push it to a registry (a warehouse), pull it onto any machine (a kitchen), run it as a container (dinner is served).

SECTION 05

Command Cheat Sheet

The 20 commands you will use 95% of the time. Every command supports `--help` when you need more.

Images - the blueprints

Command	What it does
docker pull IMAGE	Download an image from a registry
docker images	List all images on your machine
docker build -t NAME:TAG .	Build an image from a Dockerfile in the current folder
docker rmi IMAGE	Delete an image
docker tag SRC TARGET	Give an existing image another name
docker push IMAGE	Upload an image to a registry

Containers - the running things

Command	What it does
docker run IMAGE	Create and start a container from an image
docker run -d IMAGE	Run in the background (detached)
docker run -it IMAGE bash	Run interactively, into a shell inside the container
docker run -p 8080:80 IMAGE	Map host port 8080 -> container port 80
docker run -v /local:/data IMAGE	Mount a host folder into the container
docker ps	List running containers
docker ps -a	List all containers (including stopped)
docker stop CONTAINER	Gracefully stop a container
docker start CONTAINER	Start a stopped container again
docker rm CONTAINER	Delete a container
docker logs CONTAINER	See what the container has printed
docker exec -it CONTAINER bash	Open a shell inside a running container

Cleanup

Command	What it does
docker system prune	Delete stopped containers, unused networks, dangling images
docker system prune -a	Same, plus all unused images (aggressive - read the prompt)

Most common mistake. People confuse `docker stop` with `docker rm`. *stop* pauses a container - it still takes disk space. *rm* deletes it. Use `docker ps -a` to see the graveyard of stopped containers you forgot to remove.

SECTION 06

The Dockerfile, Explained

A Dockerfile is a plain text file - a recipe for your image. Every line is one instruction. Here is a complete real one with annotations.

```
# 1. Base image - the starting point
FROM python:3.12-slim

# 2. Set working directory inside the container
WORKDIR /app

# 3. Copy dependency list first (caching trick)
COPY requirements.txt .

# 4. Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the rest of your app
COPY . .

# 6. Document which port the app listens on
EXPOSE 8080

# 7. The command to run when the container starts
CMD ["python", "app.py"]
```

What each instruction means

Instruction	Meaning
FROM	Start from a base image. `python:3.12-slim` = Python 3.12 on minimal Linux. Every Dockerfile starts with FROM.
WORKDIR	Set the working directory for all following commands. Creates it if missing.
COPY	Copy files from your machine into the image. `COPY src dest`.
RUN	Execute a command at build time. The result is baked into the image.
EXPOSE	Document which port the container uses. Does not publish it - that is `docker run -p`.
CMD	The default command when the container starts. Only the last CMD counts.

The layer caching trick. Put *rarely-changing* instructions first (base image, dependency install). Put *often-changing* instructions last (your app code). Docker rebuilds everything from the first changed layer onward. COPY requirements.txt + RUN pip install BEFORE COPY . . saves minutes on every rebuild.

SECTION 07

Volumes & Networks

Containers are ephemeral - delete one, and everything inside it disappears. Volumes and networks solve the two things containers can't do alone: **keep data** and **talk to each other**.

Volumes - persistent data

A volume is a folder on your host machine that Docker mounts into the container. Anything written there survives the container.

Side	What it means
Host folder	/home/you/data - survives everything, lives on your laptop
Container folder	/data - looks like a normal folder inside; writes land on host
Command	docker run -v /home/you/data:/data myapp

Networks - containers talking

By default, containers are isolated. A network lets containers on the same network reach each other by name instead of IP address.

Container	Can reach
web container	http://api:8080 - finds the api container by name
api container	db:5432 - finds the database container by name
db container	internal only - not exposed to the host

When to use what. Use a **volume** for anything you want to persist (database files, uploads, logs). Use a **network** when two or more containers need to talk to each other by a stable name.

SECTION 08

Docker Compose

Real apps have multiple pieces: a web server, an API, a database, maybe a cache. Running `docker run` five times with the right flags every time is painful. **Compose** lets you describe the whole stack in one file.

```
# docker-compose.yml - the whole stack in one file
version: "3.9"

services:
  web:
    build: ./web
    ports:
      - "80:80"
    depends_on:
      - api

  api:
    build: ./api
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Then just two commands

Command	What it does
docker compose up -d	Start everything in the background
docker compose down	Stop and remove everything

Why Compose matters

- **Reproducible.** Your teammate clones the repo, runs `docker compose up`, and gets the exact same environment you have.
- **Version-controlled.** The `docker-compose.yml` file lives in git, so the whole stack is documented as code.
- **No manual setup.** Ports, environment variables, volumes, dependencies - all in one place.
- **Same file, local and server.** The same Compose file runs your dev environment and a simple production deploy.

> **When to graduate.** Compose is perfect for local development and small deployments. For real production across many machines, look at **Kubernetes** - but learn Compose first. It is the same mental model, scaled up.

SECTION 09

Best Practices & What to Read Next

The 10 rules beginners should internalize

- **One container, one job.** A container should do one thing. Don't run nginx and postgres in the same container.
- **Use small base images.** `alpine` and `-slim` variants are tiny. Faster to pull, safer to run.
- **Use .dockerignore.** Like .gitignore - excludes unneeded files from your build context. Big speedup.
- **Don't run as root inside the container.** Use USER in the Dockerfile. Containers that run as root are a security risk.
- **Pin your versions.** `python:3.12.1-slim`, not `python:latest`. latest moves under you.
- **Use layers wisely.** Combine RUN commands where sensible; put slow-changing instructions first.
- **Never bake secrets into an image.** API keys, passwords - use environment variables or a secrets manager, not COPY or ENV.
- **Use volumes for persistent data.** Never store a database inside a container without a volume - data loss is guaranteed.
- **Clean up.** Run `docker system prune` periodically. Containers pile up.
- **Read the Dockerfile out loud.** If a line seems redundant, it probably is. Simplicity is a feature.

Common beginner mistakes

Mistake	What to do instead
docker run with 10 flags every time	Write a docker-compose.yml and use docker compose up
Rebuilding whole image on every code change	Order the Dockerfile so COPY . . comes last
Storing data inside a container	Mount a volume. Containers are ephemeral.
Using latest tags in production	Pin exact versions. Predictability beats convenience.
Running as root	Add USER appuser after installing dependencies.
Big images from scratch	Start from alpine or -slim variants.

The one thing to remember. Docker solves exactly one problem: *"It works on my machine"* becomes *"it works the same everywhere."* Everything else - images, containers, volumes, networks, Compose - exists to make that sentence true.

DOCKER - BEGINNER GUIDE

Ship the machine, not just the app.

10 pages - beginner level